Topic 5: The this Pointer and Static Members

5.1 The this Pointer

- Every non-static member function receives an implicit pointer named this pointing to the calling object
- Used to disambiguate when a parameter has the same name as a member variable
- Used to return the current object from a function (enables method chaining)
- Type: ClassName* const this -- you can read it but cannot make it point elsewhere

```
class Counter {
    int count;
public:
    Counter(int count) : count(count) {}  // 'this->count' vs parameter 'count'

    Counter& increment() {
        count++;
        return *this;   // return current object for chaining
    }
    void show() { cout << count << endl; }
};

// Usage: c.increment().increment().show();
```

5.2 Static Data Members

- A static data member belongs to the class itself, not to any individual object
- Shared by all objects -- there is only one copy regardless of how many objects exist
- Must be defined outside the class: int ClassName::member = 0;
- Use case: counting how many objects have been created

```
class Car {
    static int count;   // declaration inside class
    string model;
public:
    Car(string m) : model(m) { count++; }
    ~Car()                   { count--; }
    static int getCount()    { return count; }
};

int Car::count = 0;  // definition outside class
```

5.3 Static Member Functions

- A static member function can be called without creating an object: ClassName::function()
- Cannot access non-static members or use this pointer -- only static members are in scope
- Useful for factory functions, utility helpers, or accessing static counters

```
int main() {
    Car c1("Toyota"); Car c2("Honda");
    cout << Car::getCount() << endl;  // prints 2
}
```